Scalable Fine-Grained eBPF Isolation through Hybrid Policy Verification

Royal Dsouza
MCA Program
St. Aloysius University, Miri, India

***Abstract*—Multi-tenant cloud systems use extended Berkeley Packet Filter (eBPF) programs for networking, observability, and security while sharing a Linux kernel. Kernel-verifier acceptance and tenant-specific policy compliance are related but distinct properties. This paper studies a selective-analysis design in which a conservative abstract policy analyzer decides straightforward cases and sends unresolved cases to a symbolic verifier. The project first established a reproducible baseline and measured two controlled workloads: a compiled-feature micro-corpus (E1) and a synthetic family with 1–64 feasible symbolic paths (E2). The E1 evidence audit passed with qualifications; the measurements suggest that instruction count alone is a weak predictor of end-to-end cost in the tested micro-corpus, while E2 shows increasing cost across its controlled path family. The project also implemented a three-valued abstract analyzer, a 24-program validation corpus, and a Phase 5 execution protocol. However, the repository's final results audit did not accept E3–E5 as completed experiments because the required execution artifacts were not present in the audited repository state. We therefore treat the often-cited 50% discharge and 45.53% savings figures as unverified report values, not established findings. The paper contributes a conservative design, a reproducible evaluation methodology, baseline observations, and an explicit account of the evidence still needed to support an end-to-end performance claim.**

***Index Terms*—eBPF, multi-tenant cloud security, eBPF isolation, symbolic execution, abstract analysis, policy verification, KRAKENGUARD**

I. Introduction

Cloud infrastructure increasingly consolidates multiple workloads on shared Linux hosts. This multi-tenant model improves utilization, but it also means that security mechanisms operating inside the kernel must distinguish what one tenant is authorized to do from what another tenant is allowed to do. Extended Berkeley Packet Filter (eBPF) provides a programmable interface for networking, tracing, observability, and security, making it attractive for cloud-native infrastructure. At the same time, its kernel-level execution model makes isolation and policy enforcement important security concerns.

The Linux eBPF verifier provides a fundamental safety gate, but verifier safety and fine-grained tenant policy compliance are different questions. A program may satisfy verifier safety requirements while still violating a tenant-specific policy governing helper calls, maps, memory, return values, or interactions with other programs. Recent systems have therefore explored fine-grained policy analysis and related verifier-assurance mechanisms, while broader studies characterize the Linux eBPF runtime and its operational context [1]. KRAKENGUARD, for example, uses symbolic analysis to reason about fine-grained eBPF isolation policies and identifies symbolic path and expression growth as an important scalability concern [2]. Other recent work addresses verifier precision through proof-guided refinement, verifier-resilient execution, verifier testing, tenant virtualization, and resource isolation [3]–[7].

This work focuses on a narrower question within that broader multi-tenant security problem: whether expensive symbolic policy verification can be invoked selectively rather than universally. The central research question is:

> Can fine-grained eBPF isolation policies be checked with predictable cost by combining inexpensive abstract analysis with selective symbolic execution?

The proposed approach places a lightweight abstract policy analyzer before symbolic verification. The analyzer uses a conservative three-valued decision model: SAFE, VIOLATION, or UNKNOWN. A proven safe program is accepted, a proven violation is rejected, and an unresolved program is forwarded to the frozen symbolic verifier. The design is intentionally fail-closed: lack of proof is not treated as safety.

The work does not replace the Linux verifier or KRAKENGUARD. It investigates a selective policy-analysis workflow and documents both its prototype design and the evidence required to evaluate its performance credibly.

The paper makes four contributions:

1) We characterize the cost behavior of a frozen KRAKENGUARD configuration using controlled program-feature and symbolic-path experiments.
2) We design a conservative abstract policy-analysis stage with explicit SAFE, VIOLATION, and UNKNOWN outcomes.
3) We implement a conservative three-valued abstract analyzer and specify how it integrates with symbolic fallback.
4) We audit the strength and limits of the available evidence, separating measured baseline observations from Phase 5 results that are not independently reproducible from the audited repository state.

II. Background and Motivation

A. eBPF in Multi-Tenant Infrastructure

eBPF allows user-supplied programs to execute at selected kernel hooks. In cloud infrastructure, XDP is one relevant hook because it enables high-performance packet processing close to the network receive path. The same programmability

that makes eBPF useful also creates an isolation requirement: a tenant-specific program must remain within the authority defined for that tenant.

The security problem can be expressed as a policy decision over an eBPF program P and policy Π:

$$V(P, \Pi) \in \{\text{COMPLIANT}, \text{VIOLATION}\}.$$

The policy may constrain operations such as helper calls, map accesses, memory behavior, or return values. The important distinction is that policy compliance is workload- and tenant-specific, whereas verifier safety is a more general property of whether the program is safe to execute under the kernel's verifier rules.

B. Symbolic Policy Verification

KRAKENGUARD provides a relevant baseline for fine-grained eBPF isolation. Its verification pipeline lifts eBPF bytecode into an intermediate representation, resolves helpers and other symbols, constructs a verification harness, and uses KLEE-based symbolic execution to explore policy-relevant behavior [2]. This makes symbolic execution useful for cases where the policy decision depends on values that cannot be determined by simple local inspection.

The drawback is that symbolic execution can require exploration of multiple feasible execution paths. If a program has P feasible paths, the symbolic workload can increase as P increases. The exact relationship is workload dependent; this paper does not assume a universal asymptotic complexity. Instead, E2 experimentally controls path growth in a synthetic family.

C. Why a Hybrid Approach?

A verifier that sends every program directly to symbolic execution pays the complete symbolic pipeline cost even when the policy decision is already obvious from the program's compiled structure. A conservative abstract analyzer can sometimes prove that a forbidden action is unreachable or that all reachable policy-relevant operations are compliant.

The key design principle is therefore:

$$H(P, \Pi) = \begin{cases} \text{ACCEPT}, & A(P, \Pi) = \text{SAFE}, \\ \text{REJECT}, & A(P, \Pi) = \text{VIOLATION}, \\ V_{\text{sym}}(P, \Pi), & A(P, \Pi) = \text{UNKNOWN}. \end{cases} \quad (1)$$

Here, A is the abstract analyzer and V_{sym} is the authoritative symbolic verifier. The hybrid method does not claim that symbolic execution becomes intrinsically faster. Its potential benefit comes from avoiding symbolic execution entirely for programs that the abstract stage can decide.

III. Related Work

Recent eBPF research covers several dimensions of the security and isolation problem. KRAKENGUARD directly addresses fine-grained eBPF isolation using symbolic analysis and serves as the primary baseline for this study [2]. BCF applies proof-guided abstraction refinement to improve verifier precision while constraining kernel complexity [3]. Approximation Enforced Execution investigates runtime enforcement for untrusted Linux kernel extensions and changes the assumptions around verifier enforcement [4]. Veritas/SpecCheck uses specification-based testing to detect eBPF verifier misbehavior [5].

Multi-tenant execution isolation is also being investigated independently of policy verification. vBPF explores eBPF virtualization and tenant/state isolation [6], while PeeR addresses scheduling and resource isolation for latency-critical eBPF applications [7]. BPF Token changes the delegation model for BPF operations through user-namespace-bound BPF filesystems [8]. SeaBee addresses protection of security-critical eBPF infrastructure against privileged tampering [9].

These systems motivate the broader multi-tenant security context but do not make them interchangeable baselines for the specific policy-verification experiment in this paper. In particular, the present work evaluates a hybrid verification strategy against a frozen KRAKENGUARD policy-analysis configuration rather than proposing a new BPF namespace, virtualization layer, scheduler, or verifier.

IV. Research Problem and Hypotheses

The research problem is to reduce unnecessary symbolic policy-verification work without converting uncertainty into an unsafe decision.

The primary research question is:

> Can inexpensive abstract policy analysis discharge policy-safe eBPF programs before symbolic execution, while selectively forwarding unresolved programs to symbolic verification?

The evaluation is divided into two logical questions. First, can the abstract stage safely classify a useful subset of programs? Second, does avoiding symbolic verification reduce complete end-to-end verification cost?

The Phase 5 hypotheses are evaluation targets, not findings until the full execution matrix is available and independently checked:

- **H5-C: Correctness.** Abstractly discharged final decisions agree with the frozen symbolic reference on the validation corpus.
- **H5-COV: Selective discharge.** A measurable subset of the corpus can be classified without symbolic fallback.
- **H5-P: End-to-end cost.** Hybrid verification reduces complete verification time when abstract-analysis cost plus any required fallback cost is lower than symbolic-only verification.
- **H5-S: Fallback scaling.** Fallback amount and cost can be characterized as symbolic path complexity increases.

V. Hybrid Verification Design

A. Architecture

The system is intentionally simple:

The analyzer operates on the compiled eBPF object and its control-flow graph rather than relying only on source-level

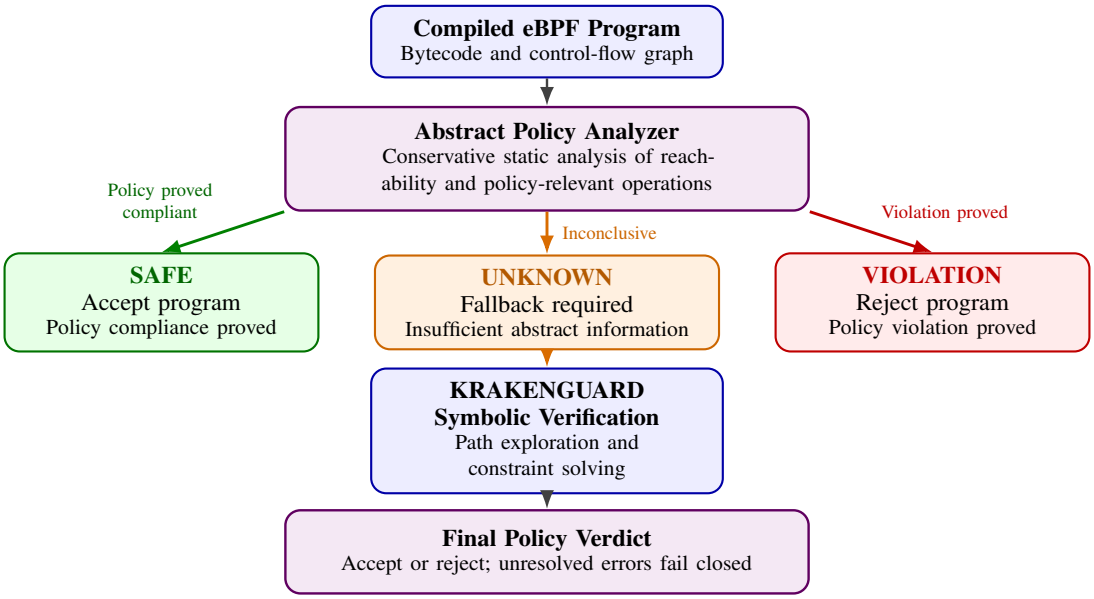


Fig. 1. Hybrid fine-grained eBPF policy-verification pipeline. The abstract analyzer accepts or rejects only when it can establish a conclusive result; unresolved cases are sent to KRAKENGUARD, and verification failures do not imply acceptance.

C syntax. This distinction follows from the semantics of the BPF instruction set and the Linux toolchain, in which source constructs may be optimized or transformed before verification [10], [11]. It tracks reachability, selected integer facts, policy conditions, helper access, map access, memory ranges, return values, and analysis status.

B. Three-Valued Abstract Decision

The analyzer returns exactly one of three abstract decisions:

- SAFE: every reachable abstract state is proven policy compliant.
- VIOLATION: a policy-forbidden action is proven reachable.
- UNKNOWN: the available abstract domain cannot prove compliance or violation.

The distinction between UNKNOWN and SAFE is fundamental. Unsupported instructions, unknown helper identities, unresolved map identity or access mode, insufficient memory proofs, and unresolved branch conditions conservatively produce UNKNOWN. A syntactic occurrence of a forbidden operation is not sufficient for VIOLATION unless its reachability is established.

C. Fail-Closed Semantics

The analyzer is designed around a one-sided safety requirement: an abstract analysis failure must reduce precision, not increase authority. Therefore:

$$\text{UNKNOWN} \not\Rightarrow \text{SAFE}.$$

Only the symbolic fallback can resolve an unresolved case. This design also makes the hybrid system compatible with the role of KRAKENGUARD as the authoritative reference for the experiments.

VI. Experimental Methodology

A. Frozen Baseline and Environment

All experiments use a frozen KRAKENGUARD revision:

`e7bd84005b304c5a10efcdb04914d1882b3cccf7`.

The fixed policy used by E1 and Phase 5 has SHA-256:

`270403272d736ae7aee2ceda3bf8d088b6ac0cb476bb99ce02`

The experimental environment is:

- Linux kernel 7.2.3-arch1-2, x86_64;
- AMD Ryzen 5 5600H, 12 logical cores;
- approximately 7.1 GiB RAM;
- Clang/LLVM 22.1.8;
- BPF target with `-mcpu=v1`;
- optimization `-O2 -g`;
- XDP hook type;
- Docker-based KRAKENGUARD artifact with immutable image digest `sha256:9633a6922518589803a4c9b8123d054`

The compiler flags were `-target bpf -mcpu=v1 -D__TARGET_ARCH_x86 -O2 -g` with the required system include path.

B. E1: Program Complexity Scaling

E1 evaluates how end-to-end KRAKENGUARD cost changes across eight synthetic XDP micro-programs while keeping policy and environment fixed. Each program was run for two warmups and seven measured repetitions, for 72 total executions.

The compiled corpus ranged from a 2-instruction minimal XDP program to a 20-instruction program combining branches, helpers, and a map. The measured median wall times were:

TABLE I
E1 MEDIAN END-TO-END VERIFICATION TIME.

Program	Instructions	Paths	Median (ms)
P0	2	1	681.82
P1	5	1	679.87
P2	4	1	677.55
P3	18	1	672.43
P4	6	1	688.57
P5	11	1	692.90
P6	17	1	686.70
P7	20	2	707.61

TABLE II
E2 VERIFICATION COST VERSUS OBSERVED FEASIBLE SYMBOLIC PATHS.

Paths	Median (ms)	Mean (ms)
1	746.14	757.28
2	749.93	752.64
4	762.77	770.22
8	789.88	789.19
16	853.02	854.79
32	973.44	972.45
64	1231.16	1266.86

P0–P3 remained nearly flat despite a large increase in compiled instruction count. P4 exhibited one genuine timing spike, which was retained rather than removed. E1 therefore indicates that raw instruction count alone does not explain the observed end-to-end cost in this small regime. Feature-dependent symbolic behavior and fixed pipeline overhead are separate dimensions.

C. E2: Controlled Symbolic Path Scaling

E2 isolates feasible symbolic path growth more directly. The controlled program family produced 1, 2, 4, 8, 16, 32, and 64 feasible KLEE paths. Each level used two warmups and seven measured repetitions, for 63 total executions and 49 measured observations.

The median increased by approximately 65.0% from one path to 64 paths. A linear fit in the tested regime was approximately:

$$T(P) \approx 740 \text{ ms} + 7.6 \text{ ms} \cdot P,$$

with $R^2 > 0.999$ for the measured points. This is an empirical model for the tested family, not a claim of universal asymptotic complexity. E2 establishes the motivation for reducing the number of programs that require symbolic verification.

D. Phase 5 Corpus

The Phase 5 validation corpus contains 24 programs divided equally into four categories:

- **A:** abstractly provable compliant, 6 programs;
- **B:** abstractly uncertain but symbolically compliant, 6 programs;
- **C:** abstractly provable violation, 6 programs;
- **D:** abstractly uncertain and symbolically violating, 6 programs.

This structure separates easy cases from cases that intentionally require the symbolic fallback.

E. E3: Selective Discharge and Correctness

E3 is specified to evaluate whether the abstract stage can classify a useful subset and whether discharged decisions agree with the frozen symbolic reference. The protocol calls for two warmups and seven measured repetitions for each of 24 programs, producing 216 planned executions.

The primary endpoint is:

$$D = \frac{N_{\text{SAFE}} + N_{\text{VIOLATION}}}{N_{\text{total}}}.$$

A correct hybrid decision is defined relative to the authoritative KRAKENGUARD result. The experiment requires zero false SAFE and zero false VIOLATION decisions; false UNKNOWN is acceptable because it invokes the fallback.

F. E4: End-to-End Performance

E4 compares two complete pipelines:

$$T_{\text{symbolic}} = T_{\text{KRAKENGUARD}}, \tag{2}$$

$$T_{\text{hybrid}} = \begin{cases} T_{\text{abstract}}, & \text{if discharged,} \\ T_{\text{abstract}} + T_{\text{fallback}}, & \text{otherwise.} \end{cases} \tag{3}$$

The E4 protocol specifies both conditions for each of 24 programs, with two warmups and seven measured repetitions per condition, for $24 \times 2 \times 9 = 432$ planned executions. This protocol total must not be confused with verified evidence that all runs completed. Conditions were interleaved using deterministic randomization with seed 42. The primary endpoint is paired per-program median end-to-end wall time.

G. E5: Fallback Scaling

E5 is designed to examine hybrid behavior across symbolic path strata. The corpus design identifies 1, 2, 4, 8, 16, and 32 as reachable strata; 64 was marked unreachable for eligible Phase 5 programs. These are design/summary claims and require the underlying execution artifacts before being treated as independently verified observations.

The experiment reports abstract time, fallback time, total hybrid time, and fallback fraction. It does not assume that the abstract analyzer accelerates symbolic execution itself. Its intended benefit is elimination of unnecessary symbolic invocations.

VII. RESULTS AND EVIDENCE STATUS

A. E1: Program Features Do Not Explain Cost on Their Own

The E1 dataset contains eight programs, each with two warmup and seven measured executions. The independent evidence audit found 72 rows, unique run identifiers, consistent policy and baseline identifiers, and exact agreement between the recorded CSV measurements and JSON aggregates. It therefore passed the dataset-integrity check, with qualifications about the meaning of several instrumentation fields.

The median wall times in Table I lie in a narrow range despite the increase in compiled instruction count. This is

TABLE III
EVIDENCE STATUS AT THE TIME OF MANUSCRIPT REVISION.

Item	Status	Interpretation
E1	Audited, qualified	Dataset integrity passed; several metrics and compiler effects need qualification.
E2	Recorded baseline	Controlled path family and timing summaries are present in project records.
Analyzer	Implemented	Three-valued outcomes and fail-closed fallback are specified.
E3	Not independently verified	Claimed discharge counts require raw execution evidence.
E4	Not independently verified	Claimed timing savings require all paired runs and recomputed aggregates.
E5	Not independently verified	Claimed strata summaries require reproducible source telemetry.
Katran	Unsuccessful attempt	The run terminated before producing a valid verification result.

evidence only about the tested micro-corpus and pipeline. It does not establish that program size is irrelevant in general. The audit also found that one source-level branch had been optimized away by Clang, that the reported basic-block count was heuristic, and that the field named `verifier_states` represented KLEE's explored-path statistic. Those qualifications limit how strongly the table can be interpreted.

B. E2: Controlled Path Growth

The E2 project record reports a controlled family with 1, 2, 4, 8, 16, 32, and 64 feasible symbolic paths. The median increased from 746.14 ms at one path to 1231.16 ms at 64 paths, an increase of approximately 65%. A linear fit of roughly

$$T(P) \approx 740 + 7.6P \text{ ms}$$

summarizes the tested range. It is a descriptive fit, not a complexity theorem. The observation motivates testing whether a selective front-end can reduce the number of programs that require symbolic execution.

C. Phase 5: Prototype and Evaluation Status

The repository contains the three-valued abstract analyzer, a 24-program policy-directed corpus, correction and fallback validation records, a frozen execution specification, and the E3–E5 experimental protocol. The protocol defines three questions: how many cases can be discharged without symbolic execution (E3), whether hybrid execution reduces complete end-to-end cost (E4), and how fallback behaves across reachable path strata (E5).

However, the final independent results audit did not accept E3–E5 as completed experiments in the audited repository state. It found that the claimed execution commit and the required raw result sets were unavailable for independent recomputation. The summaries below are therefore evidence about the project's intended evaluation design, not accepted experimental outcomes.

Older project summaries report a 12/24 abstract-discharge rate and 45.53% E4 savings. Because the required execution artifacts were not available to the final audit, neither figure is presented as an established result here. The same applies to claims of zero correctness mismatches across the full Phase 5 matrix. These figures can be reinstated as results only after the immutable execution artifacts are recovered, run accounting is checked, and all summaries are regenerated from raw telemetry.

VIII. DISCUSSION

The project provides a useful example of why systems-security papers need to separate a plausible optimization from a demonstrated one. E1's audited measurements show that, for this small compiled micro-corpus, instruction count alone does not account for the end-to-end verification time. The audit's compiler and instrumentation qualifications are equally important: the branch structure in the object differed from source-level intent, and some counters were proxies rather than direct measurements.

E2 addresses a different dimension by varying the feasible symbolic path count in a controlled program family. Its recorded timing trend supports the motivation for selective analysis within the tested range. It does not show that every real-world workload will follow the same relationship, nor does it by itself demonstrate that a hybrid design will be faster. That second claim depends on the fraction of cases discharged, the cost of abstract analysis, and the cost of fallback.

The abstract analyzer is designed around this distinction. It returns SAFE, VIOLATION, or UNKNOWN; it does not treat uncertainty as permission. The three-valued interface gives the system a way to preserve the symbolic verifier as the authority for cases that the abstract domain cannot settle. This is the central design rationale and is independent of whether the current Phase 5 performance hypothesis is ultimately confirmed.

The project's reported Phase 5 summaries would, if verified, suggest a trade-off that is worth testing: conclusively discharged cases could avoid symbolic execution, while fallback cases would pay the abstract-stage overhead before receiving the reference verdict. But the raw E3/E4/E5 artifacts were not available to the final independent results audit. The 50% discharge and 45.53% savings values are therefore not used as evidence for this paper's conclusions. They should be treated as testable claims pending artifact recovery or a new execution campaign.

The engineering record also includes an unsuccessful attempt to run a real-world Katran object through the frozen verifier. Because the attempt did not produce an authoritative policy result, it cannot support a performance comparison. It instead points to a practical requirement for future work: establish compatibility and obtain a valid baseline verdict before interpreting timing data from a production-oriented program.

IX. THREATS TO VALIDITY AND LIMITATIONS

Several limitations bound the conclusions.

A. Corpus Scope

The Phase 5 corpus contains 24 synthetic, policy-directed XDP programs. It is useful for controlled evaluation of the decision categories but is not representative of all production eBPF workloads. Large programs, complex parsers, large maps, tail calls, loops, and application-specific policies may exhibit different behavior.

B. Reference Oracle

Correctness is measured against the frozen KRAKENGUARD configuration. Agreement with a reference implementation establishes consistency with that implementation; it does not independently prove universal correctness of KRAKENGUARD.

C. Environment

All measurements were collected on one host, one kernel version, one compiler version, one XDP configuration, one frozen KRAKENGUARD revision, and one fixed policy. Results may differ on other systems.

D. Path Range

E2 experimentally covers 1–64 feasible paths. E5 reaches only 1–32 paths for eligible Phase 5 programs; the 64-path Phase 5 stratum was unreachable. No claim is made about behavior beyond these ranges.

E. Measurement Scope

Wall time is measured for the complete verification workflow. Client-process CPU time is not treated as equivalent to verifier wall time. Memory scaling and detailed solver-time scaling are not used as primary conclusions because they were not consistently exposed by the frozen pipeline.

F. Generalization

The 50.0% discharge rate is a property of the evaluated 24-program corpus. It must not be interpreted as the fraction of arbitrary eBPF programs that can be discharged. Similarly, the 45.53% savings figure is an observed corpus-level result and not a universal performance guarantee.

X. Reproducibility and Provenance

The repository specifies the major controls needed to reproduce the configuration: KRAKENGUARD revision, policy hash, compiler, kernel, architecture, hook type, compiler flags, container image digest, corpus metadata, and expected raw execution artifacts. For Phase 5, however, the final independent results audit did not find the complete execution artifacts needed to recompute the reported results.

The repository records Phase 5 design, correction-validation, fallback-validation, and execution-gate artifacts. The gate review authorized execution; it did not establish that the performance experiments had run. The independent results audit explicitly found that the claimed execution commit and E3/E4/E5 raw result sets were absent from the audited repository state. Consequently, the E3 discharge counts, E4 timing summaries, and E5 scaling table elsewhere in older result summaries are not treated here as completed, independently reproducible experiments. A provenance record or README summary is not a substitute for raw execution rows and a reproducible aggregation path.

This evidence boundary is consequential: the manuscript reports E1 as an audited baseline with qualifications, treats E2 as a controlled baseline result from the project records, and describes E3–E5 as an implemented design and planned evaluation whose claimed outcome numbers require artifact recovery and independent recomputation before publication.

TABLE IV
Hybrid decision semantics and required action.

Abstract result	Evidence available	Required action
Safe	Policy compliance proved	Accept
Violation	Policy violation proved	Reject
Unknown	Neither proof established	Symbolic fallback
Analysis error	No trusted result	Fail closed
Timeout	No completed proof	Do not accept

XI. Formal Decision Model and Safety Invariant

Let P denote a compiled eBPF program, Π a fixed policy, and $\mathcal{R}(P)$ the set of reachable program states under the supported execution model. Let $\mathrm{Bad}_\Pi(s)$ identify a state whose policy-relevant behavior violates Π. The target property is:

$$\forall s \in \mathcal{R}(P), \quad \neg\, \mathrm{Bad}_\Pi(s).$$

This formulation is intentionally limited to the modeled program, policy semantics, and reachable states considered by the verifier. It does not imply isolation across arbitrary programs, maps, links, credentials, or tenants unless those interactions are explicitly included in the model.

The abstract analyzer computes an over-approximation $\widehat{\mathcal{R}}(P)$ of reachable states. A Safe result is justified only when the over-approximation excludes all states violating the policy. A Violation result requires a reachable abstract witness that is sufficiently precise to establish the violation under the analyzer's supported semantics. If either proof obligation cannot be established, the result is Unknown. This is the standard conservative role of abstraction: the abstract domain may lose precision, but that loss must not be interpreted as evidence of compliance [12].

A. Decision Table

Table IV summarizes the intended operational semantics. The table describes the design contract, not a claim that every possible eBPF instruction or helper is modeled.

This distinction matters because the costs of the three abstract outcomes are asymmetric. A conservative Unknown may add runtime to an otherwise compliant program, but an incorrect Safe result can admit a policy violation. Accordingly, the design optimizes only where the analyzer has sufficient evidence. It does not optimize by reinterpreting missing information as permission.

XII. Cost Model and Break-Even Conditions

The hybrid workflow is useful only when the cost of abstract analysis plus the cost of any required fallback is lower than direct symbolic verification. For a program P_i, let A_i be abstract-analysis time, S_i be symbolic-only time, and F_i be the incremental symbolic fallback time when the abstract result is UNKNOWN. The hybrid cost is

$$H_i = A_i + I_i F_i,$$

where $I_i = 1$ for UNKNOWN and $I_i = 0$ for a conclusively discharged program. The per-program time difference is

$$\Delta_i = H_i - S_i.$$

A negative value favors the hybrid workflow; a positive value indicates overhead. For a discharged program, $H_i = A_i$, so the potential saving is approximately $S_i - A_i$. For an unresolved program, $H_i = A_i + F_i$, which may exceed S_i if fallback repeats most of the original work.

For a corpus with discharge fraction d, mean abstract cost $\bar{A}$, mean symbolic cost $\bar{S}$, and mean fallback cost $\bar{F}$ among unresolved programs, a simplified expected-cost expression is

$$\mathbb{E}[H] = \bar{A} + (1 - d)\bar{F}.$$

This expression is descriptive rather than a universal performance model: the means may differ by category, path count, and program structure, and the measured E4 endpoint uses paired per-program medians. Nevertheless, it makes the break-even requirement explicit:

$$\bar{A} + (1 - d)\bar{F} < \bar{S}.$$

A high discharge rate is not sufficient by itself if abstract analysis is expensive; similarly, a cheap abstract stage may not help when nearly every case falls back. The experimental evaluation must therefore report discharge coverage and end-to-end cost together.

XIII. Algorithmic Workflow

Algorithm **??** expresses the routing policy independently of implementation details. The key property is that only conclusive abstract outcomes terminate the workflow.

The implementation should record the abstract decision, the reason for uncertainty, whether fallback was invoked, the fallback's authoritative policy result, elapsed time for each stage, and identifiers for the program and policy artifacts. Such records make it possible to distinguish an optimization win from a failed or incomplete verification attempt. A timeout, malformed response, missing policy verdict, or analysis error must remain a distinct status; it must not be collapsed into a successful verification result.

XIV. Measurement Protocol and Statistical Interpretation

The experiments use repeated measurements because wall-clock runtime can vary with host scheduling, I/O, process startup, and other system activity. Warmup executions are separated from measured repetitions. The E4 design interleaves the symbolic-only and hybrid conditions with deterministic randomization so that one condition is not always measured first. The unit of comparison is the program, not an individual repeated run treated as an independent program.

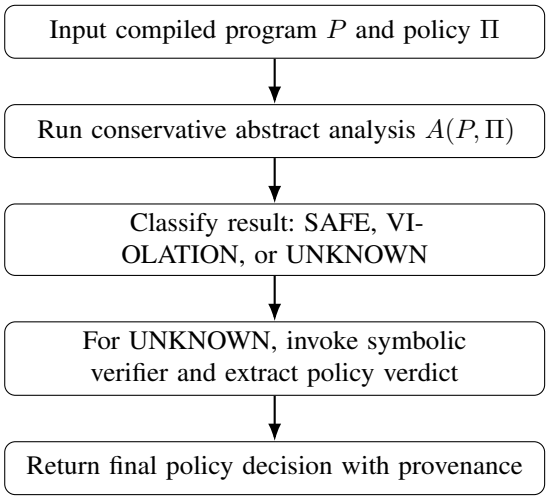


Fig. 2. Operational workflow. Conclusive abstract outcomes terminate at the routing stage; only unresolved cases require symbolic fallback.

For program i, let $\widetilde{S}_i$ and $\widetilde{H}_i$ be the medians across the seven measured repetitions. The paired difference and ratio are

$$d_i = \widetilde{H}_i - \widetilde{S}_i, \qquad r_i = \frac{\widetilde{H}_i}{\widetilde{S}_i}.$$

The reported aggregate savings should be computed from the predeclared aggregate endpoint, with the denominator and aggregation order stated explicitly. A ratio of aggregate medians is not generally equal to the median of per-program ratios. For this reason, the manuscript distinguishes the overall 45.53% reported result from the category-specific savings and recommends retaining the raw telemetry alongside summary tables.

The 24-program corpus is small and intentionally structured around four semantic categories. The Wilson interval reported for a 12/24 discharge proportion describes uncertainty in a binomial proportion under its statistical assumptions; it does not make the constructed corpus a random sample of production eBPF programs. The interval is therefore useful as a descriptive uncertainty measure, not as a basis for estimating arbitrary production coverage.

A complete reproduction package should preserve raw outputs, run identifiers, source and object hashes, policy hash, baseline revision, container digest, compiler flags, kernel and architecture metadata, status codes, and aggregation scripts. These controls help independent reviewers determine whether summary numbers can be regenerated from the recorded measurements rather than relying on manually transcribed tables.

XV. Relationship to Prior Systems

The hybrid architecture draws on the general idea of abstract interpretation, where a tractable abstract domain summarizes concrete program behavior while conservatively losing precision when necessary [12]. Symbolic execution complements this approach by exploring path conditions and solving constraints; KLEE is a well-known symbolic execution engine for LLVM programs [13]. In this paper, the two techniques serve different operational roles: abstraction handles cases with simple proofs, while symbolic execution remains the fallback for cases that depend on path-sensitive values.

Prior eBPF confinement work addresses related but distinct boundaries. bpfbox demonstrates precise process confinement using eBPF-based policy enforcement [14]; BPFContain studies container-security confinement using eBPF [15]. Research on cross-container attacks documents how overly permissive eBPF capabilities can create isolation risks in shared-kernel environments [16]. These works motivate policy expressiveness and threat modeling but are not direct performance baselines for the present hybrid verifier.

The Linux kernel documentation specifies eBPF instruction behavior, verifier concepts, helpers, maps, and program types; the BPF ISA is also standardized in IETF RFC 9669 [10], [11]. The runtime literature provides complementary context for how the in-kernel execution environment, helper interface, and verifier interact [1]. Those references are important for implementation and reproducibility because source-level C does not fully determine the compiled instruction stream. The compiler may remove branches, fold predicates, or alter control flow, so the abstract analyzer must reason about the compiled artifact and the experiment must record compiler settings.

XVI. Practical Implications and Deployment Boundaries

The measured result suggests a potential use case for verification gateways that repeatedly check policy-specific eBPF programs before loading or delegating them. Such a gateway could attempt a fast abstract proof and invoke symbolic analysis only when needed. This is a hypothesis for deployment design, not evidence that the current prototype is ready to operate as a production admission controller.

A deployment would require additional engineering beyond the evaluated corpus: explicit versioning of policy semantics, compatibility testing across kernel and compiler releases, resource limits for fallback, robust timeout handling, authenticated artifact provenance, isolation of the verifier process, and safe treatment of verifier crashes. It would also need to define how policy changes interact with already loaded programs, maps, links, and delegated capabilities. These are not measured by E1–E5 and remain outside the current contribution.

The distinction between kernel safety and application policy remains important. The hybrid analyzer is a policy-analysis front-end and does not replace the kernel's verifier, permissions, namespaces, cgroups, capabilities, or other isolation controls. A positive result in the evaluated program-policy model is not a proof of end-to-end tenant isolation. The appropriate conclusion is narrower: selective abstract discharge can reduce the measured verification work for a subset of the tested policy cases while preserving fallback for uncertainty.

XVII. Research Evolution and Engineering Record

The direction of this project changed as the literature and implementation evidence accumulated. That history is relevant to the contribution because it explains why the paper does not claim a new isolation primitive or a replacement for the kernel verifier.

A. Why the Research Question Was Narrowed

The initial motivation was broad: improve isolation for eBPF workloads in multi-tenant Linux systems. The literature review showed that a number of adjacent mechanisms already address important parts of this space. KRAKENGUARD directly targets fine-grained policy isolation; BCF focuses on verifier precision; BPF Token changes delegated access; vBPF explores tenant-oriented virtualization; PeeR addresses execution scheduling; and bpfbox/BPFContain provide confinement-oriented approaches. These systems differ in goals and threat models, so they are not interchangeable baselines. Their existence nevertheless rules out presenting generic eBPF isolation as the novelty.

The surviving problem is narrower and measurable: symbolic policy verification can be expensive, while some policy decisions may be decidable by a cheaper abstract pass. The design question is whether a conservative front-end can reduce the number of symbolic invocations without changing the final decision. This framing makes the intended contribution falsifiable. If the abstract pass rarely discharges programs, or if its cost plus fallback exceeds direct symbolic execution, the optimization hypothesis is not supported.

XVIII. Threat Model, Assumptions, and Non-Goals

The target environment is a shared Linux host on which multiple workloads may submit or operate eBPF programs. The immediate concern is policy compliance at load or admission time: whether the compiled program's modeled behavior respects a tenant-specific policy governing selected helper calls, map operations, memory behavior, and return-dependent policy conditions. The verifier's policy decision is made before relying on the program for the intended workload.

The design assumes that the compiled object, policy definition, and symbolic-verifier configuration are bound to stable identifiers or hashes for the duration of an experiment. It also assumes that the kernel verifier remains enabled and retains responsibility for its existing safety checks. The hybrid layer answers an additional policy question; it does not grant kernel execution permission by itself.

The following are non-goals of the current prototype:

- replacing or weakening the Linux verifier;
- proving complete isolation across arbitrary programs, maps, links, credentials, and tenant lifecycle events;
- solving all possible eBPF instruction, helper, map, loop, tail-call, or memory semantics;

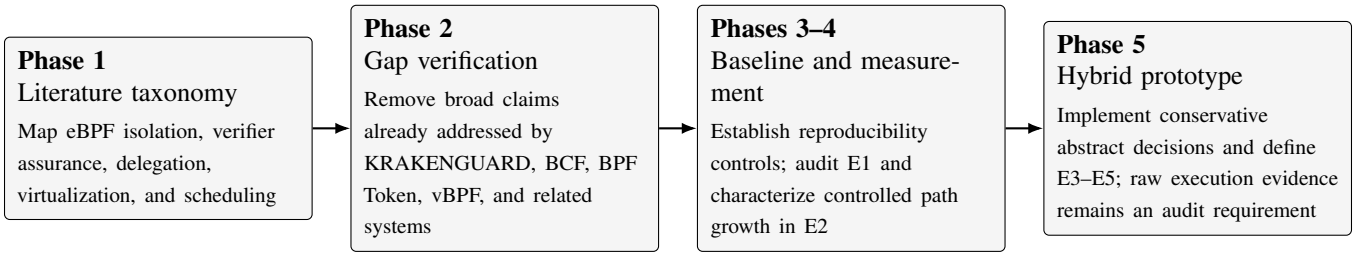


Fig. 3. Research progression and the evidence boundary. The phases represent the project's documented evolution, not a claim that every planned experiment has completed.

TABLE V
RESEARCH DECISIONS AND WHAT THE AVAILABLE EVIDENCE SUPPORTS.

Stage	Work recorded in the project	Decision carried into this paper	Evidence boundary
Literature review	Taxonomy of eBPF confinement, verifier analysis, delegation, virtualization, runtime enforcement, and scheduling	Avoid claiming generic multi-tenant isolation or verifier testing as a new gap	Literature review motivates scope; it does not prove implementation performance
Gap verification	Comparison with KRAKENGUARD, BCF, AEE, Veritas/SpecCheck, BPF Token, vBPF, PeeR, bpfbox, and BPFContain	Focus on the cost of fine-grained policy verification and selective analysis	Novelty remains a scoped positioning claim, not a proof that no concurrent work exists
Baseline infrastructure	Frozen policy, controlled program corpus, run identifiers, environment metadata, and machine-readable aggregation	Treat reproducibility as part of the research result rather than a final packaging task	Infrastructure alone is not a successful benchmark
E1 and E2	Compiled-feature and controlled-path experiments; E1 has an independent data-integrity audit with qualifications	Separate program size from path-sensitive analysis cost	E1 metrics have documented qualifications; E2 is bounded to its synthetic family
Hybrid prototype	Three-valued abstract analysis, fail-closed uncertainty, a 24-program corpus, and an execution protocol	Keep symbolic fallback authoritative whenever the abstract proof is incomplete	The full Phase 5 performance matrix must be recoverable and independently recomputed
Supporting tooling	ProcessScope evaluation and a real-world Katran attempt	Keep telemetry and workload integration as supporting engineering, not the central contribution	ProcessScope is not the thesis contribution; the Katran attempt did not yield a valid verification result

- providing a production-ready admission controller or a complete multi-tenant runtime;
- claiming universal runtime, memory, or solver complexity from a small synthetic corpus.

This boundary prevents a common category error: a sound policy decision for a modeled program is not equivalent to a proof of end-to-end tenant isolation. Cross-program interference, object lifecycle, delegated capabilities, policy revocation, and runtime scheduling remain relevant system-level concerns, but they are not established by the current experiments.

A. Failure Semantics

A verification service must distinguish a negative policy decision from an inability to reach a decision. The prototype therefore treats a proven policy violation, an unresolved abstract result, a timeout, a malformed response, and an internal analysis error as different outcomes. A violation can be rejected; an unknown case can be forwarded to the symbolic engine; a timeout or error cannot be accepted as proof of compliance. This is stricter than simply interpreting a process exit code as a security verdict.

The fallback decision must be extracted from the authoritative conditional-policy result produced by the symbolic verifier. Generic transport-level success, a top-level response flag, or a process that exits without a valid policy artifact is insufficient. This distinction was an explicit engineering correction in the project and should remain part of the implementation contract.

XIX. IMPLEMENTATION DETAILS AND ANALYZER BOUNDARIES

The analyzer's abstract state contains reachability, selected integer facts, policy predicates, helper identities, map identities and access modes, memory facts, return-value information, and an analysis status. It propagates these facts over the compiled control-flow graph. A branch is pruned only when its predicate is established as true or false; otherwise both successors remain reachable. At control-flow joins, conflicting facts are weakened to a less precise representation.

The implementation is deliberately incomplete in the formal sense: it supports a subset of eBPF instruction and policy semantics rather than proving correctness for all of eBPF. Its safe operating contract is therefore conditional on that supported subset. Unknown helper identifiers, unresolved map identity, unsupported map update/delete behavior, insufficient memory proof, unsupported instructions, and unresolved branches must not be silently treated as compliant. They produce UNKNOWNor a hard analysis failure, which prevents the front-end from converting a lack of understanding into an authorization.

Return-value tracking matters because policy conditions may depend on helper results or branch-dependent values. A constant return can make a later condition decidable; a symbolic return may leave both branches reachable. This was one of the lessons carried forward from the earlier policy experiments: policy evaluation cannot be reduced to scanning the object for a forbidden helper name. Reachability and conditional semantics matter.

A. Compiler Effects and the Unit of Analysis

The unit of analysis is the compiled eBPF object, not the source-level C file. A compiler can fold conditions, remove branches, simplify arithmetic, and alter the control-flow graph. The E1 audit found a concrete example: a source conditional in P2 was optimized into branchless code under the selected Clang configuration. The observed binary, rather than the source intention, determines the paths available to the verifier.

For reproducibility, source and object hashes, compiler version, target CPU, optimization flags, architecture definitions, include paths, and policy identity should be recorded together. If only source files are preserved, later toolchain changes can silently alter the tested program. Conversely, a disassembly is useful evidence but is not a substitute for preserving the exact binary when byte-level reproduction is required.

XX. Measurement Lessons and Reproducibility

The project's baseline work exposed several measurement details that materially affect interpretation.

First, wall-clock time, client-process CPU time, solver time, and peak memory are different quantities. A client-side `process_time()` measurement does not account for all CPU consumed by a persistent verifier service, its symbolic executor, or its solver. Likewise, a memory sample taken from a long-running container cannot automatically be attributed to a single verification request. The manuscript therefore treats end-to-end wall time as the primary endpoint for the current performance question and does not infer memory or solver-time results where the instrumentation did not expose them reliably.

Second, internal counters need precise names. The E1 audit found that a field called `verifier_states` was populated from KLEE's `explored paths` statistic. In this paper, such a metric should be called explored/completed symbolic paths, not generic verifier states. Similarly, a basic-block count generated by the heuristic $1 +$ branch count is not a control-flow-graph measurement. Metrics that are derived heuristically can still be useful for debugging, but should not be presented as directly observed structural properties.

Third, performance outliers should be retained unless a predeclared rule establishes that a run is invalid. E1's timing spike was preserved, and the available evidence did not establish a single cause. Removing it after seeing the result would bias the estimate. The same principle applies to failed runs, timeouts, and disagreements: each must be recorded and categorized rather than silently rerun or excluded.

Fourth, every aggregate should be reproducible from raw telemetry. A complete run record should include a unique run identifier, program and object hashes, policy hash, baseline revision, environment metadata, condition, repetition index, execution order, verdict, status, wall time, and available symbolic telemetry. The aggregation script should emit the published tables from these records. A README table or manually transcribed summary is useful for orientation but does not independently establish that an experiment ran.

A. Evidence Recovery as a Publication Gate

The Phase 5 protocol defines 216 E3 executions and 432 E4 executions, but a protocol count is not the same as an observed run count. To publish E3–E5 as completed experiments, the project needs an immutable execution commit, a verifiable relationship to the authorized execution-gate parent, all raw rows, unique run IDs, consistent artifact hashes, and scripts that regenerate the published summaries. E4 in particular needs both conditions for every program, all seven measured repetitions after warmup, and a recomputable paired analysis. E5 must be regenerated from actual telemetry and must not invent a result for an unreachable path stratum.

This requirement is not merely administrative. It protects the paper from a mismatch between an attractive aggregate and the data that supposedly supports it. If the original execution artifacts cannot be recovered, the appropriate path is to rerun the approved matrix under the frozen protocol and publish the new evidence with full provenance, not to reconstruct missing raw values from summary statistics.

XXI. Supporting Infrastructure and Real-World Validation

ProcessScope was assessed as supporting experimental infrastructure: an eBPF tracepoint collector and user-space processing path can help observe process execution, but this capability does not itself implement the policy-verification contribution. It should therefore be described as instrumentation or an auxiliary prototype rather than as a central result of the thesis.

The project also attempted to exercise the workflow on Katran, a production-oriented XDP load balancer. The selected source revision was compiled with Clang 13 into an eBPF object, but the direct KRAKENGUARD run terminated with a failure status before a valid policy result was obtained. The attempt is useful because it exposes compatibility and integration issues between a real-world object and the frozen verifier environment. It is not a successful production-workload evaluation and must not be counted as one.

A stronger real-world evaluation will require a compatible toolchain and verifier configuration, a documented mapping from the workload to the supported policy, and a successful reference-verifier result before timing comparisons are meaningful. If the baseline rejects or crashes before producing an authoritative policy decision, its elapsed time cannot be interpreted as a valid hybrid-versus-symbolic performance measurement.

XXII. Additional Validity Considerations

The corpus is synthetic and policy-directed. This is a strength for controlled tests because the expected decision categories are explicit, but it limits external validity. The 24 programs were constructed to contain both abstractly decidable cases and cases requiring symbolic reasoning; a production corpus may have a different distribution. The measured discharge fraction, if later validated, should therefore be reported as corpus-specific.

The reference oracle is a frozen KRAKENGUARD configuration. Differential agreement can establish consistency with that reference under the selected policy and environment, but it does not prove that the reference itself is free of defects. A stronger study could add manually reviewed counterexamples, independent policy fixtures, or a second reference implementation. Such an extension should preserve the distinction between reference agreement and absolute correctness.

The environment also constrains generalization. The baseline uses one kernel, compiler, architecture, hook type, policy, and verifier revision. Different compiler optimizations can change branch structure; different kernels can expose different helper semantics; and different workloads can change the relative cost of startup, lifting, symbolic execution, and fallback. Replication across more than one environment is necessary before describing the approach as generally scalable.

Finally, the project measures analysis cost, not the operational cost of deploying a complete multi-tenant security service. Policy updates, concurrent submissions, cancellation, resource quotas, cache invalidation, revocation, object lifecycle, and interaction with already-loaded programs require separate design and evaluation. These questions are important future work, but they should not be conflated with the narrow cost reduction question evaluated here.

XXIII. Conclusion and Future Work

This project began with a broad interest in multi-tenant eBPF confinement and narrowed the thesis question after reviewing newer work on fine-grained isolation, verifier precision, delegation, virtualization, and scheduling. That review matters: a new namespace, a generic LSM policy, or another verifier-testing tool would overlap with existing systems and would not by itself establish a defensible contribution. The surviving question is more specific: can a conservative abstract front-end avoid some symbolic policy-verification work without changing the final security decision?

The strongest completed empirical evidence currently available is the baseline characterization. E1 passed an independent data-integrity audit with qualifications, including compiler transformations that changed the intended branch structure and instrumentation fields that must not be over-interpreted. E2's controlled family showed increasing end-to-end cost from one to 64 feasible paths. Those observations motivate selective analysis, but they do not alone demonstrate that the hybrid system improves end-to-end performance.

The project also produced an abstract analyzer, a structured 24-program corpus, fail-closed handling for unsupported or unresolved semantics, a frozen policy and environment specification, and an execution protocol for E3–E5. The currently audited repository does not, however, contain the raw Phase 5 execution artifacts needed to independently substantiate the frequently reported 50% discharge rate and 45.53% timing savings. Those numbers are therefore excluded from the paper's established findings. Recovering the immutable execution tree, checking every run, and regenerating the aggregates is the immediate prerequisite to making a performance claim.

This distinction does not make the project less useful; it makes the paper more defensible. The contribution at this stage is a carefully bounded hybrid design, a baseline analysis, and a reproducible evaluation plan with explicit failure semantics. Future work should complete the Phase 5 evidence audit, expand the corpus beyond synthetic XDP programs, measure additional hooks and policy forms, and investigate targeted refinement for cases that return UNKNOWN. A separate real-world Katran attempt also exposed a compatibility boundary: the attempted run terminated before producing a valid verification result, so it is an engineering lesson rather than evidence of successful production-workload evaluation.

Acknowledgment

The author thanks the maintainers and authors of the open-source eBPF analysis infrastructure used as experimental baselines. The experiments and analysis reported here are intended as a research evaluation and not as a production security guarantee.

References

[1] B. Gbadamosi, L. Leonardi, T. Pulls, T. H
oiland-J
orgensen, S. Ferlin-Reiter, S. Sorce, and A. Brunstr
om, "The ebpf runtime in the linux kernel," *arXiv preprint arXiv:2410.00026*, 2024. [Online]. Available: https://arxiv.org/abs/2410.00026

[2] J. Patel, L. G. Buhl-Nielsen, A. Ghosn, and M. Kogias, "Krakenguard: Towards fine-grained ebpf isolation," in *Proceedings of the 19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 2026)*, 2026. [Online]. Available: https://www.usenix.org/conference/nsdi26/presentation/patel

[3] H. Sun and Z. Su, "Prove it to the kernel: Precise extension analysis via proof-guided abstraction refinement," in *Proceedings of the 31st ACM Symposium on Operating Systems Principles (SOSP 2025)*, 2025. [Online]. Available: https://doi.org/10.1145/3731569.3764796

[4] ——, "Approximation enforced execution of untrusted linux kernel extensions," in *USENIX Security Symposium*, 2025. [Online]. Available: https://www.usenix.org/conference/usenixsecurity25/presentation/sun-hao

[5] T. Lyu, K. K. Dwivedi, T. Bourgeat, M. Payer, M. Xu, and S. Kashyap, "ebpf misbehavior detection: Fuzzing with a specification-based oracle," in *Proceedings of the 31st ACM Symposium on Operating Systems Principles*, 2025. [Online]. Available: https://doi.org/10.1145/3731569.3764797

[6] J. Zhang, X. Song, D. Du, Y. Xia, B. Zang, and H. Chen, "Virtualizing ebpf with late-binding," in *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 2026)*, 2026. [Online]. Available: https://www.usenix.org/conference/osdi26/presentation/zhang-jing

[7] J. Carin, B. Holmes, W. Wang, A. Bhardwaj, and M. Ghobadi, "Peer: First-class scheduling for latency-critical ebpf applications," in *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 2026)*, 2026. [Online]. Available: https://www.usenix.org/conference/osdi26/presentation/carin

[8] Linux Kernel Maintainers, “Bpf system call documentation,” Linux Kernel Documentation, 2024–2026. [Online]. Available: https://docs.kernel.org/userspace-api/ebpf/syscall.html

[9] National Security Agency, “Seabee: Security enhanced architecture for ebpf,” Open-source project, 2025–2026. [Online]. Available: https://github.com/NationalSecurityAgency/seabee

[10] Linux Kernel Documentation, “Bpf documentation,” Linux Kernel Documentation, 2026. [Online]. Available: https://docs.kernel.org/bpf/

[11] D. Thaler, “BPF instruction set architecture,” RFC 9669, 2024. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9669.html

[12] P. Cousot and R. Cousot, “Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints,” in *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL)*, 1977, pp. 238–252. [Online]. Available: https://doi.org/10.1145/512950.512973

[13] C. Cadar, D. Dunbar, and D. Engler, “KLEE: Unassisted and automatic generation of high-coverage tests for complex systems programs,” in *Proceedings of the 8th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2008. [Online]. Available: https://www.usenix.org/conference/osdi-08/klee-unassisted-and-automatic-generation-high-coverage-tests-complex-systems

[14] W. Findlay, A. Somayaji, and D. Barrera, “bpfbox: Simple precise process confinement with ebpf,” in *Proceedings of the 2020 Cloud Computing Security Workshop (CCSW)*, 2020, pp. 107–120. [Online]. Available: https://doi.org/10.1145/3411495.3421358

[15] W. Findlay, D. Barrera, and A. Somayaji, “Bpfcontain: Fixing the soft underbelly of container security,” in *arXiv preprint arXiv:2102.06972*, 2021. [Online]. Available: https://arxiv.org/abs/2102.06972

[16] Y. He, R. Guo, Y. Xing, X. Che, K. Sun, Z. Liu, K. Xu, and Q. Li, “Cross container attacks: The bewildered ebpf on clouds,” in *Proceedings of the 32nd USENIX Security Symposium*, 2023, pp. 5971–5988. [Online]. Available: https://www.usenix.org/conference/usenixsecurity23/presentation/he